

COURSE PROJECT REPORT: ARFOM-DB

Airline Reservation and Flight Operations Management System

Course:	Database Management Systems (DBMS)
Candidate Name:	Sanjyit Suresh Kumar (Roll No. 25WU0102243)
Section / Roll:	AIML Whales - Serial #18
Academic Year:	2026 - 2027
Submission:	Final Comprehensive Report (5 Marks)

1. Abstract

Airline flight operations and reservation networks demand zero-downtime, ACID-compliant relational architectures capable of guaranteeing seat exclusivity, customer itinerary tracking, and baggage reconciliation across complex route networks. ARFOM-DB is a normalized relational database management system engineered to manage the complete lifecycle of airline operations. Built upon strict Third Normal Form (3NF) relational decomposition, the database models 10 core tables and analytical views encompassing airport hubs, aircraft configurations, seat maps, scheduled flight legs, customer profiles, booking transactions, e-tickets, payments, check-in custody, baggage manifests, and cancellation penalty ledgers. The system features engine-enforced domain, entity, and referential constraints - including a composite unique invariant preventing physical double-booking. A multi-tier web application and Python REST dispatch API interface directly with the relational engine, enabling live record viewing, atomic insertions, and referential-safe deletions with sub-millisecond database reflection.

2. Problem Statement & System Objectives

Commercial aviation operates under narrow financial margins where seat allocation errors, uncollected baggage surcharges, or untracked cancellations directly impact profitability and regulatory compliance. Legacy reservation approaches often rely on decentralized or partially normalized flat files, leading to three critical database failure modes:

- Concurrency Contention & Double-Booking: Simultaneous reservations writing to identical flight-seat pairs.
- Relational Redundancy: Repeated entry of passenger profiles, route distances, and airframe capacities leading to update and deletion anomalies.
- Lack of Auditability: Inability to trace financial transactions from initial booking to check-in or cancellation.

3. Objectives and Scope

- Design a complete Entity-Relationship model capturing 1:1, 1:N, and decomposed M:N relationships.
- Achieve strict 3NF and BCNF normalization across all tables.
- Enforce declarative integrity constraints (PK, FK RESTRICT/CASCADE, CHECK, UNIQUE).
- Implement multi-table analytical queries for flight load factor, revenue corridor density, and cargo logistics.
- Develop a fully connected, responsive User Interface demonstrating live record viewing, insertion, and deletion.

4. System Architecture & Relational Topology

- Infrastructure Module: airports, routes, aircraft, seats
- Operations Module: flights, checkins, baggage
- Commercial Module: passengers, bookings, tickets (junction entity resolving M:N)
- Audit & Finance Module: payments, cancellations

5. Relational Normalization Analysis

- 1NF Compliance: All attribute values are strictly atomic. Multi-valued contacts or baggage items are fully decomposed into dedicated tuples.
- 2NF Compliance: All non-prime attributes are fully functionally dependent on the primary key. Composite primary keys (e.g. seats.aircraft_id, seat_number) contain no partial dependencies.
- 3NF Compliance: No transitive functional dependencies exist among non-prime attributes ($X \rightarrow Y$ holds only where X

is a superkey). Dynamic values like excess baggage fees are computed via operational views.

6. Verification & Negative Constraint Test Results

Test ID	Constraint Evaluated	Test Input	Engine Behavior	Status
TC-01	CHECK (weight_kg >= 0)	weight_kg = -5.50	Engine aborts write in 0.8ms	PASS
TC-02	UNIQUE (flight_id, seat_id)	Flight 1, Seat 1 twice	Duplicate key rejected	PASS
TC-03	CHECK (origin != dest)	DEL -> DEL	Write aborted	PASS
TC-04	FOREIGN KEY ON DELETE RESTRICT	DELETE Airport 'DEL'	Deletion blocked	PASS
TC-05	ACID INSERT Transaction	Passenger: V. Malhotra	Row ID 61 allocated in 2.1ms	PASS

7. User Interface Implementation & CRUD Operations

The application layer connects directly to the relational database through a zero-dependency Python REST server:

- Viewing Module: Live data grid with multi-table selector, search filters, column sorting, and pagination.
- Insertion Module: Form validation with 1-click viva presets and real-time row count increment (+1).
- Deletion Module: Pre-deletion inspection with referential safety warnings and instant row count decrement (-1).
- Before & After Comparator: Side-by-side 3-column state transition proving live storage engine reflection.

8. Conclusion & Repository Deliverables

ARFOM-DB provides an enterprise-ready, mathematically rigorous relational foundation for airline reservation and fleet dispatching. All source code, DDL/DML scripts, slide decks, and test harnesses are submitted in the public repository:

- Repository URL: <https://github.com/Sanjyit-S/DBMS-Course-Project>
- Presentation-I: Problem Description PDF & Scope Definition
- Presentation-II: Interactive ER Studio, DDL Scripts & Analytical Queries
- Presentation-III: Live UI Source Code, Slide Deck & CRUD Verification
- Project-Report: Comprehensive 16-Chapter Technical Documentation